

SOUL FORGE

Developer Documentation

A stealth-platformer game made using python and Pygame

Made by: 1820251084 宋理、1820251088 颜键杨

Contents

1. Project Overview.....	4
1.1. Tech Stack	4
2. File and Folder Structure.....	4
3. Running from Source	5
4. Game Architecture: The State Machine	5
4.1. States	5
4.2. Notable Transition Logic	6
5. Core Systems.....	6
5.1. Player	6
5.1.1. Key Constants	6
5.1.2. Movement Mechanics	7
5.2. Camera	7
5.3. Level Building.....	7
5.3.1. Warden Terminals	8
5.3.2. Ending Terminals	8
5.4. Terminals & Hacking Minigame	8
5.4.1. Terminal (hackTerminal.py).....	8
5.4.2. Hacking Minigame (hackMinigame.py)	9
5.5. Doors	9
5.6. Base Enemies.....	9
5.6.1. Pathfinding	10
5.7. The Warden	10
5.7.1. Dialogue System	10
5.7.2. Chase AI – Grid-Based A*	10
5.8. Lore & Journal.....	11
5.9. Save System.....	11
5.9.1. Save File Location	11
5.9.2. Autosave.....	12
5.10. Audio System.....	12
5.10.1. SFX	12
5.10.2. BGM & Speed Shift.....	12
5.11. Sprite Sheets & Animations	12
6. Asset Loading & resource_path() function.....	13
7. Building a .exe application	13

7.1.	main.spec	13
7.2.	build.bat	14
7.3.	Producing a New Build	14
7.4.	Testing a Fresh Build	14
8.	Known Quirks, Bugs & Gotchas	14
9.	Extra Notes	15
9.1.	Language & Translation	15
9.2.	AI Usage	15

1. Project Overview

Soul Forge is a 2D stealth-platformer built in Python using Pygame. The player controls Marquette Verne, the '73rd Soul', navigating a large multi-room map, avoiding patrolling enemies, hacking terminals to open doors, collecting lore fragments, and eventually confronting the final boss, The Warden Gerard Gatewald, who offers a narrative choice that leads to two separate conclusions to the game.

The code base is organized as a single-process Pygame application: one main loop in `main.py` drives a state machine, while each gameplay system (player movement, enemies, the boss, terminals, audio, sprites, saving) lives in their own module and is driven from said loop.

1.1. Tech Stack

- Python 3 (developed against a project-specific virtual environment)
- Pygame – rendering, input, audio primitives, rects and collisions
- NumPy – used only inside `audioManager.py` for on-the-fly BGM resample (the alarm speed-up effect) as well as `warden.py` for grey scaling an image
- `pyInstaller` – Used to package the whole game into a standalone Windows .exe for distribution
- Bosca Ceoil – Used to make the music and the sound effects of the game

2. File and Folder Structure

File/Folder	Responsibility
<code>main.py</code>	Entry point. Owns the game loop, the state machine, UI/menu rendering, and wiring between all other systems
<code>player.py</code>	Player physics: movement, gravity, jumping, coyote time, wall slide and wall jump, mantling, animations
<code>enemy.py</code>	Patrol-guard AI: Patrol/Chase/Returning states, line-of-sight detection, BFS waypoint pathfinding
<code>warden.py</code>	Boss character: dialogue system, A* pathfinding/chase logic, defeat pose, sprite rendering
<code>buildLevel.py</code>	Declarative level data — rooms, platform geometry, terminal/door/enemy/lore-key placement — and the <code>build_level()</code> factory that assembles a fresh level
<code>hackTerminal.py</code>	Terminal object: hacking state, countdown timer, alarm behavior, sprite-based visual states, save/load
<code>hackMinigame.py</code>	The actual hacking mini-game: a small grid puzzle (find key, avoid dummy tiles, reach exit)
<code>door.py</code>	Doors that open once their required terminal(s) are hacked, with a delay and smooth slide animation
<code>camera.py</code>	Room-snapping camera with a smooth lerp transition between rooms
<code>checkpointManager.py</code>	Trivial checkpoint zone — a rect that triggers an autosave on entry (logic lives in <code>main.py</code>)

lore.py	Journal/lore system: collectible LoreKey objects, the JournalManager that tracks unlocked chapters, and all chapter text
saveManager.py	Persistent save data (3 slots) as JSON, plus the shared Button UI class used throughout the menus
audioManager.py	SFX + music playback, including a custom pitch/speed-shift effect on the background music during alarms
spriteSheet.py	Generic sprite-sheet slicer and a simple frame-based Animation helper used by player/enemy/terminal
assets/	Houses all the used assets, each in their own folder (sprites/ and music/ for each)

Note: Every module that loads an image or sound file from assets/ needs the resource_path() helper described in Section 6 — this is the single most common thing to remember when adding new content.

3. Running from Source

Dependencies (required installs):

```
pip install pygame numpy
```

Run the game:

```
python main.py
```

The game window is set to be 1000 by 750 and targets 60 FPS (see SCREEN_WIDTH / SCREEN_HEIGHT / FPS near the top of main.py)

4. Game Architecture: The State Machine

main.py is build around a single global variable, current_state, whose value is one of a fixed set of string constants defined near the top of the file. The main loop dispatches update logic and draw logic based on this value. There is no formal state-machine class – transitions are plain assignments to current_state scattered through event handling and update logic.

4.1. States

File/Folder	Responsibility
MENU	Title screen
LEVEL_SELECT	Save slot selector
PLAYING	Normal gameplay — movement, enemies, terminals, checkpoints
PAUSED	Pause overlay; game logic frozen
HACKING	Inside a terminal's mini-game. Remembers which state to return to via hackReturnState
GAME_OVER	Player was caught by an enemy or the Warden
JOURNAL	Full-screen lore journal, reachable from the pause menu
READING	A single just-unlocked lore chapter, shown immediately on pickup
WARDEN_DIALOGUE	Typewriter-style dialogue box over the boss arena (used both for the intro speech and the defeat speech)
WARDEN_INTRO	Scripted fade/teleport sequence that drops the player into the boss arena

WARDEN_OUTRO	Scripted flash/teleport sequence after the Warden is defeated, moving both characters to the ending room
ENDING_CHOICE	The two ending terminals are live; hacking either one commits the player to that ending
ENDING_FADE	Fade to black after a choice is made
ENDING_EPILOGUE	Typewriter-style epilogue text unique to each ending
ENDING	Final results screen (time, ending label)

4.2. Notable Transition Logic

- `PLAYING` → `HACKING` happens on `pygame.K_e` when the player overlaps a terminal's interact zone; the state to return to is stored in `hackReturnState`, not hardcoded, precisely so the same key handler works from both `PLAYING` and `ENDING_CHOICE`.
- `HACKING` exits either manually (pressing `E` again) or automatically once a terminal's short `exitDelay` elapses after being hacked (`Terminal.updateTimer()` returns `True` on that frame) — both paths route through `hackReturnState`, so hacking an ending terminal correctly returns to `ENDING_CHOICE` rather than falling back to `PLAYING`.
- Defeating the Warden (`hackedCount >= 8` and all Warden-related lore collected) sets `current_state = WARDEN_OUTRO`, which is a short scripted sequence that ends by starting the `WARDEN_DIALOGUE` state with `warden.dialogue_lines` swapped to `DEFEAT_LINES`.
- Finishing the defeat dialogue sets `current_state = ENDING_CHOICE` and calls `enterEndingChoice()`, which fades in the two ending terminals.
- `world_timer` only advances while `current_state` is one of `PLAYING` / `HACKING` / `WARDEN_INTRO` / `WARDEN_OUTRO` — explicitly excluding `ENDING_CHOICE`, so time doesn't run during the final decision.

5. Core Systems

As mentioned earlier, the game itself uses several different core systems separated into their own modules to run the full game.

5.1. Player

`player.py` implements a fairly involved platformer controller: acceleration-based horizontal movement, gravity, coyote time, wall sliding with a speed ramp, wall jump, and wall mantling.

5.1.1. Key Constants

Constant	Value	Meaning
GRAVITY	0.6	Added to vel_y every frame
JUMP_STRENGTH	-13	Initial upward velocity on jump
SHORT_HOP_CUTOFF	-4	Releasing jump early clamps vel_y to this, enabling variable jump height
ACCEL / FRICTION	0.8 / 0.8	Horizontal acceleration and ground friction multiplier
MOVE_SPEED	5	Horizontal speed cap
COYOTE_TIME_MAX	5 frames	Grace period to still jump after walking off a ledge
MAX_FALL_SPEED / MAX_FAST_FALL_SPEED	50 / 200	Terminal velocity, normal vs. holding down to fast-fall
wallJumpForceX / wallJumpForceY	10 / -12	Impulse applied on a wall jump
maxSlideSpd / fastSlideSpd	10.0 / 25.0	Wall-slide speed cap, normal vs. fast-fall variant

5.1.2. Movement Mechanics

- Coyote time: coyoteTimer is refreshed while grounded and ticks down after leaving the ground; jump() checks coyoteTimer > 0 as an alternative to on_ground.
- Wall slide: only triggers if the player is airborne, touching a wall, falling, and holding into the wall (checkSurroundings()); the slide speed cap ramps up the longer slideTimer has been running, in three tiers (slow grip → medium → fast).
- Wall mantle: if the player's head is clear of the wall above them while sliding, mantleClimb() automatically boosts them up and over rather than sliding.
- Wall jump: applies a stronger horizontal impulse if the player is holding away from the wall (holdAway) versus a neutral-multiplier impulse otherwise, and sets a short wallLockout to prevent instantly re-grabbing the same wall.
- Collision (moveX / moveY) is swept in fixed-size sub-steps (MAX_STEP = 10px) to avoid tunneling through thin platforms at high speed.
- Animation: walkAnim plays while grounded and moving; while idle, the idle frame is squished sinusoidally (math.sin(idleTimer / 15.0)) purely as a breathing-style visual flourish rather than a real animation frame.

5.2. Camera

Camera.py is a small room-snapping camera. Each room is a fixed pygame.Rect; targetRoom() sets a target position and update() lerps the camera position toward it every frame (self.x/y += (target - x/y) * transSpeed). isTrans is True while a transition is in progress, and main.py uses this flag to freeze player input/physics and enemy AI mid-transition so nothing updates while the camera is moving between rooms.

apply(rect) is the standard “world-to-screen” conversion used by every draw() method in the codebase: it simply offsets a world-space rect by the camera's current position.

5.3. Level Building

buildLevel.py is almost entirely declarative data: a large PUZZLES list (hand-authored or randomized hacking mini-game layouts), a WAYPOINTS dictionary (a graph of named nodes with

coordinates and neighbor lists, used by Enemy for BFS pathfinding), and a long list of `pygame.Rect` platform definitions grouped by room.

`build_level(level)` assembles and returns a fresh set of gameplay objects each time it's called (on new game, on load, and on level reset):

- rooms — a list of `Rects` defining the camera's room boundaries (currently 25 rooms, indices 0–24)
- platforms — every solid collision rect in the level.
- terminals — Terminal instances built from entries in `PUZZLES` via `copy.deepcopy(PUZZLES[i])` (deep-copied so multiple terminals never share mutable puzzle state).
- doors — Door instances, each wired to a list of `requiredTerminals` that must all be hacked before it opens.
- enemies — Enemy instances placed at fixed coordinates with patrol ranges and the shared `WAYPOINTS` graph.
- loreKeys, checkpoints — collectible/save-trigger objects.
- warden — only constructed if the room list is long enough to contain the boss arena (`WARDEN_ARENA_ROOMS`); its spawn position is derived from the first arena room's rect.

5.3.1. Warden Terminals

Once the Warden is placed, 8 additional terminals with the tag `room = "warden"` are generated at fixed hand-placed positions (`wardenTerminalPositions`) but with puzzle layouts drawn via `random.sample()` from a pool of 16 harder puzzles (`PUZZLES[10:26]`). This means the specific puzzle at each Warden-arena terminal changes between runs, while their positions in the arena stay fixed.

5.3.2. Ending Terminals

The final two terminals in the list uses `room = "endingChoiceWarden"` and `room = "endingChoiceBreak"` while also having an 'infinite' time limit (set to 99999.0) so they can never expire mid-decision. `main.py` identifies them specifically by this room string (see `syncEndingTerminal()` in `main.py`) rather than by list position, which is what lets the ending-specific UI text (see Section 8) key off `t.room`.

5.4. Terminals & Hacking Minigame

5.4.1. Terminal (`hackTerminal.py`)

A Terminal wraps a `hackingMinigame` instance and adds the surrounding game-facing behavior: a countdown timer, an alarm state if that timer runs out, a short exit delay after being hacked (so the `HACKING` state doesn't snap back to gameplay instantly), and sprite-based visuals for its three states (default / alarm / hacked).

- `canInteract(playerRect)` — the player must be within the terminal's rect inflated by (40, 50) to be able to press E.

- `updateTimer(dt, isActive)` — only counts down once `hasStarted` is `True` (i.e. the player has actually opened it at least once); on hitting zero it sets `alarmTriggered` and gives a 10-second `alarmBuffer` before the alarm can clear (if later hacked).
- rotation support — a terminal can be rotated 90/-90 degrees, in which case its `rect`'s width/height are swapped at construction so collision still lines up with the rotated sprite.
- `saveDict()` / `loadDict()` — only persist timer/hacked/alarm state, not puzzle layout, since the underlying `hackingMinigame` is deterministically rebuilt from the same `PUZZLES` entry on level load.

5.4.2. Hacking Minigame (`hackMinigame.py`)

A small grid puzzle (default 6 rows × 8 cols). The player moves a token with arrow keys/WASD, must reach a key tile, then reach an exit tile while carrying the key. Certain tiles are “dummy” decoys — stepping on one for the first time applies a time penalty (`dummyPenalty`, default 5 seconds) to the parent terminal, and can trigger the alarm outright if that penalty drains the timer to zero.

Tile positions (`keyPos`, `exitPos`, `walls`, `dummies`) are either supplied explicitly per-puzzle in `PUZZLES`, or randomized via `getRandomOpenTile()` when a puzzle entry only specifies a `dummyCount`.

5.5. Doors

A Door moves smoothly between a `closedPos` and an `openPos` (a per-door offset from its closed position) once all of its requiredTerminals report `isHacked == True`. There's a configurable `openDelay` (default 1 second) between “requirements met” and the door actually starting to move, plus a fixed `moveSpd` used to interpolate `currentPos` toward `targetPos` each frame.

- `forceClose()` / `releaseForceClose()` — used by the Warden intro sequence to seal a door mid-cutscape regardless of terminal state.
- `syncToTerminalState()` — called once on level load so a door whose terminals were already hacked in a previous save snaps straight to its open position instead of animating open on load.

5.6. Base Enemies

`enemy.py` implements a three-state patrol guard: `PATROL`, `CHASE`, and `RETURNING`

State	Behaviour
PATROL	Default idle behavior (movement pattern lives outside this class in practice — <code>patrol_range/start_x</code> bound its wandering).
CHASE	Triggered by line-of-sight or <code>forceChase</code> (set by <code>main.py</code> whenever any terminal's alarm is active, so every enemy on the level goes hostile during an alarm). Moves directly toward the player if visible, or re-paths toward the last known position every <code>REPATH_INTERVAL_FRAMES</code> using the level's waypoint graph.
RETURNING	Entered once <code>chaseTimer</code> (a memory window, <code>CHASE_MEMORY_FRAMES</code> = 300 frames ≈ 5s) expires without re-

	spotting the player. Paths back toward its own spawn point; falls back to plain patrol if it can't find a route back within <code>MAX_RETURNING_FRAMES</code> .
--	---

5.6.1. Pathfinding

Enemies path along a shared, hand-authored WAYPOINTS graph (defined in `buildLevel.py`) using breadth-first search (`update_path_to()`), truncated in this document but present in `enemy.py`) rather than free-form pathfinding — nodes are looked up by proximity to a target and connected via each node's neighbor list. Nodes can be temporarily blocked (`blocked_nodes`) if an enemy repeatedly fails to make progress toward one, forcing a re-route; blocks expire after `NODE_BLOCK_DURATION_FRAMES` ($\approx 5s$).

Line-of-sight (`check_line_of_sight`) is two-part: a simple distance check for point-blank detection, and a rectangle-overlap “vision cone” (actually a rectangle, `get_vision_rect()`) extending in the direction the enemy currently faces, further gated by an actual raycast against platform geometry (`hasLOStoPlayer`, via `pygame.Rect.cline`) so enemies can't see through walls.

5.7. The Warden

`warden.py` is the single boss character, Gerard Gatewald. It combines a scripted dialogue system with a full A* pathfinding chase once the fight begins.

5.7.1. Dialogue System

`dialogue_lines` is a list of strings advanced one at a time; `tickDialogue()` reveals the current line character-by-character at `charSpeed` (60 chars/sec) and auto-advances after a short pause once fully revealed. `tickSkip()` implements a hold-to-skip gesture (holding Y for `skipHoldTime` = 1 second) that's only enabled once `canSkip` is True — this is set once the player has already seen the intro dialogue in a previous session (persisted via `SaveManager.markWardenIntroSeen()`), so returning players aren't forced to re-sit through the full intro speech.

The same dialogue system is reused for both the intro speech (`dialogue_lines` set in `__init__`) and the defeat speech (`DEFEAT_LINES` from `main.py`, swapped in after the Warden is defeated).

5.7.2. Chase AI – Grid-Based A*

Unlike the patrol enemies' hand-authored waypoint graph, the Warden paths using a full A* search (`aStarPathfinding()`) over a dynamically-generated grid built fresh around the Warden and player's current positions each replan (`findPath()`). The grid resolution is `PATH_CELL_SIZE` = 60px, capped at `PATH_MAX_GRID_CELLS` = 40 cells per axis (the cell size is enlarged if the search area would otherwise exceed that, to bound worst-case search cost). Diagonal movement is allowed but corner-cutting through two orthogonally-blocked cells is explicitly disallowed.

The raw cell-path is then smoothed (`smoothPath()`) using a greedy line-of-sight simplification: from each point, it looks as far ahead as possible for a point with a clear line (`clearLine()`, via `cline` against inflated platform rects) and jumps straight there, skipping intermediate

waypoints. Pathing re-runs every `PATH_REPLAN_FRAMES` (20 frames) or immediately if the Warden gets stuck (`stuckFrames >= 10` frames of near-zero movement while still far from its target), in which case it also nudges sideways for a short `unstuckFrames` window before resuming normal pathing.

Movement itself is direct velocity-toward-target (not physics/gravity-based like the player or patrol enemies) — the Warden effectively flies/hovers, consistent with its cosmetic hover bob (`self.hover`, purely visual, feeds the eye sprite's sine-wave bobbing in `draw()`).

5.8. Lore & Journal

`lore.py` defines every piece of collectible lore as static text in `JOURNAL_CHAPTERS` (keyed by chapter id, e.g. "ch3_2"), a `LoreKey` pickup class (a bobbing, glowing collectible), and `JournalManager`, which tracks which chapter ids have been unlocked and exposes them in a fixed reading order (`self.order`) filtered down to only unlocked entries (`get_visible_chapters()`).

Warden-arc chapters (`ch4_1`, `ch4_2`, `ch5_1`, `ch5_2` — collected as `WARDEN_LORE_IDS` in `main.py`) are treated specially: they're required (alongside hacking 8 Warden-arena terminals) to trigger the Warden's defeat, and `main.py`'s `relockWardenChapters()/restoreLoreKeys()` functions re-lock and re-sync these specific chapters whenever the level is rebuilt (e.g. on load), so a lore key doesn't stay permanently "collected" in a way that's inconsistent with the fight's completion requirements.

`NotificationManager` is a small unrelated toast-notification queue (fading text messages); it exists in `lore.py` but is currently not wired up to any call site in `main.py`.

5.9. Save System

`saveManager.py`'s `SaveManager` class persists a single JSON file with 3 fixed save slots. Each slot stores: `level`, `room`, `player_x/player_y`, a `exists` flag, the `elapsed_world_timer`, the unlocked journal chapter list, which endings have been reached, whether the Warden intro dialogue has been seen, and full snapshots of every terminal's and enemy's state (via each object's `saveDict()/loadDict()`).

5.9.1. Save File Location

`SAVE_FILE` is not a relative path — it resolves to a per-user, per-OS application data directory via a small `get_save_dir()` helper in `main.py`:

```
def get_save_dir(app_name="StealthPlatformer"):
    if sys.platform == "win32":
        base = os.environ.get("APPDATA", os.path.expanduser("~"))
    elif sys.platform == "darwin":
        base = os.path.join(os.path.expanduser("~"), "Library", "Application Support")
    else:
        base = os.environ.get("XDG_DATA_HOME", os.path.join(os.path.expanduser("~"),
".local", "share"))
    save_dir = os.path.join(base, app_name)
    os.makedirs(save_dir, exist_ok=True)
    return save_dir

SAVE_FILE = os.path.join(get_save_dir(), "save_data.json")
```

On Windows this resolves to %APPDATA%\StealthPlatformer\save_data.json. This matters specifically because the packaged .exe (Section 7) is not run from a stable, writable, or even consistent working directory — writing next to the executable (or worse, into a PyInstaller -onefile temp extraction folder, which is deleted after every run) would silently break saving. Any future save-related code must write through this same path, never a bare relative filename.

5.9.2. Autosave

Saving happens automatically in two places, both from main.py: entering a Checkpoint rect (checkCheckpoints()), which also triggers a short “SAVED” fade notification), and on the WARDEN_INTRO completion / normal game-loop exit. There is no manual save button — progress is checkpoint-driven.

5.10. Audio System

audioManager.py wraps Pygame’s mixer with two independent responsibilities: fire-and-forget SFX playback, and a single streaming background-music track that supports on-the-fly speed changes (used for the alarm-state music speed-up).

5.10.1. SFX

All SFX are preloaded once at construction into self.sounds (a name → Sound dict). playSfx() plays most sounds on any free channel from a shared pool (pygame.mixer.find_channel(True)) so overlapping SFX don't cut each other off, except "walk" footsteps, which are routed to a single dedicated walkChannel so rapid footsteps don't spam multiple channels.

5.10.2. BGM & Speed Shift

Rather than using Pygame's built-in music streaming (which has no per-track speed control), BGM is loaded fully into memory as a NumPy array (pygame.sndarray.array) and played back as a manually re-sampled Sound. setBgmSpeed() changes playback rate by resampling the array (resampleArray(), simple nearest-neighbor index selection scaled by the speed factor) and immediately restarting playback from the correct sample offset so the change is seamless rather than restarting the track from 0:00. This is what powers the “music speeds up 25% during an alarm” effect driven from main.py (alarmActive check against audioManager.bgmSpeed).

playBgm() is idempotent per track — calling it repeatedly with the same file path (as main.py does every frame based on current_state/currRoom) is a no-op after the first call, so the correct track only actually reloads when it changes.

5.11. Sprite Sheets & Animations

spriteSheet.py is a small, generic utility used by player.py, enemy.py, and hackTerminal.py. spriteSheet(filePath, frameWidth, frameHeight, scale) loads an image once and slices it into a flat list of frames (row-major order) at a fixed cell size, with optional uniform scaling. animation(frames, frameDuration, loop) is a simple frame-timer that advances through a frame list at a fixed rate and either loops or holds on the last frame.

Note: This is the file where the resource_path() fix (Section 6) was centralized — fixing image loading here automatically fixed every caller (player, enemy, terminal) without touching those files individually.

6. Asset Loading & `resource_path()` function

Any code that loads a file from disk with a plain relative path like `"assets/sprites/Chud-Sheet.png"` only works correctly when the current working directory happens to be the project root — true when running `python main.py` from the project folder in an editor, but not guaranteed once the game is packaged into a standalone `.exe` (Section 7), which may be launched by double-click from any folder.

The fix used throughout this codebase is a small helper, duplicated in each file that needs it (`spriteSheet.py`, `warden.py`, `audioManager.py`):

```
def resourcePath(relativePath):
    basePath = getattr(sys, "_MEIPASS", os.path.dirname(os.path.abspath(__file__)))
    return os.path.join(basePath, relativePath)
```

`sys._MEIPASS` is a special attribute PyInstaller injects only when running inside a bundled --onefile executable, pointing at the temp folder the bundle was extracted into. When running from plain source (no such attribute exists), it falls back to the directory containing the current `.py` file instead. Every asset path should be wrapped in this before being handed to `pygame.image.load()`, `pygame.mixer.Sound()`, etc.

Note: This must be done by hand for every new asset-loading call site that isn't already funneled through `spriteSheet.py` or `audioManager.playBgm()/playSfx()` (which already wrap it internally). Adding a raw `pygame.image.load("assets/...")` anywhere else without this wrapper will work fine from source and silently break in the packaged `.exe`.

7. Building a `.exe` application

Packaging is done with PyInstaller, driven by a checked-in `main.spec` file plus a small `build.bat` wrapper (since the project's dependencies live in an external virtual environment rather than a global Python install, the `.bat` calls that `venv`'s Python directly rather than relying on `pyinstaller` being on `PATH`).

7.1. `main.spec`

```
a = Analysis(
    ['main.py'],
    pathex=['<project folder>'],
    datas=[('<project folder>\\assets', 'assets')],
    ...
)

exe = EXE(
    pyz, a.scripts, a.binaries, a.zipfiles, a.datas, [],
    name='main',
    console=False, # windowed, no console popup
    ...
)
```

The `datas` entry is what bundles the entire `assets/` folder (including its `music/bgm` and `music/sfx` subfolders) into the executable — PyInstaller copies referenced folders recursively, so no per-subfolder entries are needed.

7.2. build.bat

```
@echo off
"<path to venv>\Scripts\python.exe" -m PyInstaller main.spec --distpath "output/dist"
-workpath "output/build" --noconfirm -clean
pause
```

- `--noconfirm` skips the interactive “overwrite existing build?” prompt, which otherwise can silently abort a rebuild when run non-interactively.
- `--clean` wipes PyInstaller's internal cache so a rebuild can't accidentally reuse a stale compiled artifact from a previous run.

7.3. Producing a New Build

Double click build.bat from the project root. The finished executable is written to output/dist/main.exe

7.4. Testing a Fresh Build

It is recommended to copy main.exe out of output/dist to an unrelated folder (e.g. the Desktop) before testing – running it from inside the project folder can mask bugs where the game accidentally still depends on loose files sitting next to the source rather than the properly bundled/redirected versions (assets via `resource_path()`, save data via `%APPDATA%`).

8. Known Quirks, Bugs & Gotchas

A running list of non-obvious behaviors and past bugs worth keeping in mind when extending this codebase:

- `hackReturnState` exists specifically so the HACKING state (entered from either PLAYING or ENDING_CHOICE) knows which state to return to on exit — both the manual-exit key handler and the automatic timer-expiry exit in the main loop must both read from this variable rather than hardcoding PLAYING, or hacking an ending terminal will incorrectly dump the player back into normal gameplay.
- The two ending terminals are identified by their room string ("endingChoiceWarden" / "endingChoiceBreak"), not by list position or object identity — any UI text or logic that needs to distinguish them (e.g. the “Become the Warden” vs “Break the Cycle” prompts, or showing “Ending Reached” instead of “Access Granted” on the hack-complete screen) should key off `activeTerminal.room.w`
- `world_timer` explicitly excludes ENDING_CHOICE (and the WARDEN_INTRO / WARDEN_OUTRO cutscenes) from ticking — if a future state needs the timer paused too, it must be added to that same exclusion tuple in the main loop.
- Any new `pygame.image.load(...)` or `pygame.mixer.Sound(...)` call must be wrapped in `resourcePath()` (Section 6) or it will silently fail once bundled into the .exe, even though it works fine from source.
- The save file's location is OS-user-specific (`%APPDATA%` on Windows), not tied to the project folder or the .exe's location — running the game from source and running the built .exe on the same machine share the same save data, by design.
- PyInstaller builds are not automatically fresh: without `--noconfirm --clean`, a build can silently fail to overwrite a stale executable if run non-interactively (e.g. from a

double-clicked .bat), leaving you debugging what looks like a code change that “didn't take.” Always check the exe's file modification timestamp against your latest source edit if a fix appears not to be working.

- Enemy AI is known to occasionally get stuck on ledges, this is a known bug which has yet to be resolved up to this point

9. Extra Notes

9.1. Language & Translation

The current rendition of the game uses English as its primary language, as seen across all of the UI elements, dialogue, and notifications. As the two developers of this game primarily uses English to read and write, we defaulted first and foremost to English when making anything in this game.

Translation to the Chinese language might be doable, though that will naturally require some extra time and a new system to allow said translation, moreover, a new file system (lang.json) would need to be made.

9.2. AI Usage

For the sake of transparency, this game does have some AI usage, namely in the coding department. In the enemy AI specifically, some code used was directly from AI. Aside from this point, AI usage was kept to a minimum.

